[M4] Examen Parcial No. 2

Marlon Hernández 15177

Repo: https://github.com/ivanhez/Examen-Parcial-No-2

Video: https://youtu.be/MUKo1M6329U

1. Considere el problema de “hacer sencillo”: dado un monto m, buscamos alcanzar dicho monto con la menor cantidad de monedas posible. Las monedas tienen las siguientes denominaciones: {1,5,10,25}. Podemos tomar todas las monedas que necesitemos de cada denominación.

   Propiedades:
   - Subestructura óptima
   - Subproblemas traslapados
   - Greedy-choice property

   Subestructura óptima:

   Sea C(m) la cantidad mínima de monedas necesarias para formar el monto m.

   Decisión:
   En cada paso decidimos qué moneda usar primero. Si el monto actual es m, las decisiones posibles son tomar una moneda d, donde:

   $$d \in \{1,5,10,25\}$$

   y además $d \leq m$.

   Subproblema generado:
   Después de tomar una moneda de valor d, hay que completar el monto restante:

   $$m - d$$

   Por tanto, el problema se reduce a encontrar la menor cantidad de monedas para el monto restante.

   Necesaria optimalidad:
   Una solución óptima para m usa primero una moneda d. Entonces el resto

de la solución debe formar $m - d$.

Para buscar contradicción, que el conjunto de monedas usado para formar $m - d$ no es óptimo, debe de existir otra forma de formar $m - d$ usando menos monedas.

Al probar la misma moneda inicial d, pero reemplazando la solución subóptima de $m - d$ por la mejor solución de $m - d$.

El resultado sería una solución para m con menos monedas que la supuesta solución óptima.

Esto contradice que la solución original era óptima. Por tanto, si una solución óptima para m empieza tomando una moneda d, debe ser una solución óptima para $m - d$.

La función de recurrencia sería la siguiente:

$$C(m) = 1 + \min_{d \in \{1,5,10,25\},\, d \leq m} C(m - d)$$

con caso base:

$$C(0) = 0$$

Esto demuestra que el problema tiene subestructura óptima.

Subproblemas traslapados:

Si resolvemos el problema recursivamente usando la recurrencia anterior, aparecen montos repetidos varias veces.

Por ejemplo, para calcular C(30), se consideran:

$$C(29), C(25), C(20), C(5)$$

Pero luego, al calcular C(29), también se puede llegar a C(24), C(19), C(4), etc. Al calcular otros caminos, vuelven a aparecer montos como C(20), C(15), C(10), C(5).

El mismo subproblema C(k) puede ser generado desde varios problemas mayores:

$$C(k+1), C(k+5), C(k+10), C(k+25)$$

Por ejemplo, C(20) puede aparecer al resolver C(21) usando una moneda de 1, al resolver C(25) usando una moneda de 5, al resolver C(30) usando una moneda de 10, o al resolver C(45) usando una moneda de 25.

Entonces una solución recursiva sin memoria resolvería varias veces los mismos montos. Esto demuestra que existen subproblemas traslapados. Por eso una solución de programación dinámica con tabla dp[m] es válida.

Subproblemas traslapados:

El algoritmo greedy tomaría siempre la moneda de mayor denominación que no exceda el monto restante. Se busca demostrar que esa decisión local pertenece a alguna solución óptima. Para estas monedas, analizamos por rangos, primero en una solución óptima:

- No puede haber 5 monedas de 1, porque se reemplazan por 1 moneda de 5.
- No puede haber 2 monedas de 5, porque se reemplazan por 1 moneda de 10.
- No puede haber 3 monedas de 10, porque se reemplazan por 25 + 5, es decir, 2 monedas en lugar de 3.
- No puede haber 2 monedas de 10 y 1 de 5, porque se reemplazan por 1 moneda de 25.

Consideramos un monto m ≥ 25, suponiendo que una solución óptima para m no usa ninguna moneda de 25. Entonces solo usa monedas de 1, 5 y 10. Pero, por las reglas anteriores, una solución óptima sin monedas de 25 no puede acumular 25 o más centavos sin contener un patrón reemplazable. Por ejemplo:

- Si se tiene 3 monedas de 10, se mejora usando 25 + 5.
- Si se tiene 2 monedas de 10 y una de 5, se mejora usando 25.
- Si se tiene muchas monedas de 1 o de 5, también se puede reemplazar por monedas más grandes.
-

Por tanto, para cualquier m ≥ 25, existe una solución óptima que usa una moneda de 25. Entonces la decisión greedy de tomar 25 es segura.

Luego queda el subproblema m - 25. Por subestructura óptima, el resto también debe resolverse óptimamente. El mismo razonamiento se aplica a los montos menores:

- Si 10 ≤ m < 25, una solución óptima debe usar una moneda de 10, si no, se usaría monedas de 5 y 1, pero dos monedas de 5 se reemplazan por una de 10.
- Si 5 ≤ m < 10, una solución óptima debe usar una moneda de 5, si no se usaría al menos cinco monedas de 1, reemplazables por una de 5.
- Si m < 5, solo queda usar monedas de 1.

Así en cada paso, la moneda más grande posible pertenece a alguna solución óptima. Por eso el problema tiene greedy-choice property.

Pseudocódigo:

```
hacer_sencillo(monto)
    monedas = [25, 10, 5, 1]
    resultado = {}

    para cada moneda en monedas:
        cantidad = monto // moneda
        resultado[moneda] = cantidad
        monto = monto % moneda

    retornar resultado
```

Código y resultados:

```python
def hacer_sencillo(monto):
    monedas = [25, 10, 5, 1]
    resultado = {}
    for moneda in monedas:
        cantidad = monto // moneda
        resultado[moneda] = cantidad
        monto =  monto % moneda
    return resultado, sum(resultado.values())

print("\nHacer sencillo")
for monto in [293, 502, 99, 42, 1337]:
    print(monto, hacer_sencillo(monto))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Hacer sencillo
293 ({25: 11, 10: 1, 5: 1, 1: 3}, 16)
502 ({25: 20, 10: 0, 5: 0, 1: 2}, 22)
99 ({25: 3, 10: 2, 5: 0, 1: 4}, 9)
42 ({25: 1, 10: 1, 5: 1, 1: 2}, 5)
1337 ({25: 53, 10: 1, 5: 0, 1: 2}, 56)
```

2. Knapsack problem fraccionado: el problema visto en clase de un ladrón con una bolsa que aguanta W unidades u de peso, y busca robar el máximo valor que pueda de una selección de n artículos, donde el artículo i tiene precio pi y hay wi u's disponibles del mismo. No es necesario robar el artículo completo; es posible tomar porciones del mismo (a nivel de u's).

   Propiedades:
   - Subestructura óptima
   - Greedy-choice property
   - Se puede modelar como una matroide ponderada

   Subestrucura óptima:

   Cada artículo i tiene

   - precio total pi
   - cantidad disponible wi
   - valor por unidad:

$$\rho_i = \frac{p_i}{w_i}$$

Decisión:
En cada paso decidimos cuánto tomar de algún artículo, para cada artículo i decidimos una cantidad:

$$0 \leq c_i \leq w_i$$

respetando que:

$$\sum_i c_i \leq W$$

$$\text{Una solución puede escribirse como:}$$

$$(c_1, c_2, \dots, c_n)$$

donde c_i representa cuántas unidades se toman del artículo i.

El valor total de la solución es:

$$\sum_i c_i \cdot \frac{p_i}{w_i}$$

Subproblema generado:
Suponiendo que en una solución óptima decidimos tomar c_j unidades del artículo j.

Entonces queda una mochila con capacidad:

$$W - c_j$$

y queda el problema de llenar esa capacidad restante usando las unidades que todavía no han sido tomadas.

Necesaria optimalidad:
Suponiendo que una solución óptima toma cierta cantidad c_j del artículo j, pero el resto de la mochila no está llenado de forma óptima.

Entonces existe otra combinación de artículos para llenar la capacidad restante $W - c_j$ con mayor valor.

Si reemplazamos el resto subóptimo por ese resto óptimo obtenemos una solución total con mayor valor que la supuesta solución óptima original.

Esto es una contradicción, por lo que después de cualquier decisión que pertenezca a una solución óptima, el subproblema restante también debe resolverse óptimamente.

Esto demuestra la subestructura óptima.

Greedy-choice property:

La decisión greedy es tomar primero la mayor cantidad posible del artículo con mayor valor por unidad.

Sea g el artículo con mayor valor:

$$\rho_g = \frac{p_g}{w_g}$$

Entonces para cualquier otro artículo j:

$$\rho_g \geq \rho_j$$

Supongamos que existe una solución óptima O que no toma todo lo posible del artículo g.

Eso significa que en O queda alguna cantidad disponible del artículo g, pero se tomó una cantidad positiva de otro artículo j con valor menor o igual.

Si se toma una pequeña cantidad μ del artículo j y se reemplaza por la misma cantidad μ del artículo g, el peso total de la mochila no cambia, porque quitamos μ unidades y agregamos μ unidades.

Pero el valor cambia así:

$$\mu\rho_g - \mu\rho_j$$

Como:

$$\rho_g \geq \rho_j$$

el valor no disminuye. Si la desigualdad es estricta, el valor aumenta.

Por tanto, cualquier solución óptima que no tome el artículo de mayor densidad puede transformarse en otra solución igual o mejor que sí toma ese artículo.

Eso demuestra que existe una solución óptima que contiene la decisión greedy. Después de tomar esa decisión, queda un subproblema de la misma forma, llenar la capacidad restante con los artículos restantes. Por subestructura óptima resolver ese subproblema óptimamente completa una solución óptima global.

Por eso el knapsack fraccionado tiene greedy-choice property.

Se puede modelar como una matroide ponderada:

Ya que se permite tomar porciones a nivel de unidades u, se puede modelar cada unidad disponible como un objeto independiente.

Sea:

$$S = \{u_{i,k}: 1 \leq i \leq n,\ 1 \leq k \leq w_i\}$$

donde $u_{i,k}$ representa la unidad k del artículo i.

Definimos la familia de independientes:

$$I = \{A \subseteq S: | A | \leq W\}$$

Un subconjunto de unidades es independiente si no excede la capacidad de la mochila.

Herencia:
Si:

$$A \in I$$

entonces:

$$| A | \leq W$$

Si B ⊆ A, entonces:

$$| B | \leq | A | \leq W$$

Por tanto:

$$B \in I$$

Se cumple la herencia.

Intercambio:
Sean A, B ∈ I y supongamos que:

$$| B |>| A |$$

Como B tiene más elementos que A, existe al menos un elemento:

$$x \in B - A$$

Además:

$$| A | +1 \leq| B |\leq W$$

Entonces:

$$A \cup \{x\} \in I$$

Se cumple el intercambio.

A cada unidad $u_{i,k}$ le asignamos el peso:

$$w(u_{i,k}) = \frac{p_i}{w_i}$$

Así al seleccionar unidades de mayor peso equivale a seleccionar primero las unidades con mayor valor por unidad. Por eso el problema se puede modelar como una matroide ponderada.

Pseudocódigo:

```
knapsack_fraccionado (W, items)
    ordenar items de mayor a menor densidad
    restante = W
```

```
total = 0
solucion = []

para cada item en items ordenados:
    si restante == 0:
        terminar
    densidad = precio / peso
    tomar = min(unidades, restante)
    valor += tomar * densidad

    agregar (item, tomar) a solucion
    total += valor
    restante -= tomar

retornar total, solucion
```

Resultado:

```
PS C:\Users\marlon.hernandez\Documents\GitHub\Examen-Parcial-No-2> py .\problemas.py
KNAPSACK FRACCIONADO

50 (240.0, [('item1', 10, 60.0, 6.0), ('item2', 20, 100.0, 5.0), ('item3', 20, 80.0, 4.0)])
15 (130.0, [('item1', 10, 100.0, 10.0), ('item2', 5, 30.0, 6.0)])
60 (295.0, [('item2', 10, 60.0, 6.0), ('item1', 20, 100.0, 5.0), ('item4', 15, 75.0, 5.0), ('item3', 15, 60.0, 4.0)])
```

3. Dado un número entero positivo n y el teclado de un Nokia 3230, que tiene dígitos del 0 al 9 asociados con cada tecla; se desea contar la cantidad total de combinaciones de n dígitos que existen, bajo las siguientes restricciones: Podemos comenzar con cualquier dígito.

    a. Podemos comenzar con cualquier dígito.
    b. El siguiente teclazo debe ser arriba, abajo, a la izquierda o a la derecha del teclazo más reciente.

c. El teclado también tiene las teclas $*$ y # que no es permitido presionar.

Propiedades:

- Subestructura óptima
- Subproblemas traslapados

Subestructura óptima:

Definimos:

$$T(k, d)$$

como la cantidad de combinaciones válidas de longitud k que terminan en el dígito d.

Decisión**:**
Para construir una secuencia de longitud k que termina en d, se debe decidir cuál fue el dígito anterior.

Ese dígito anterior debe ser un vecino válido de d.

Subproblemas generados:
Si el último dígito es d, el dígito anterior pudo haber sido cualquiera de los dígitos que se conectan con d.

Entonces para contar secuencias que terminan en d se cuentan las secuencias más cortas que terminan en cada vecino válido de d.

Dado que se puede presionar la misma tecla se modelan los vecinos de la siguiente manera:

0: {0, 8}

1: {1, 2, 4}

2: {2, 1, 3, 5}

3: {3, 2, 6}

4: {4, 1, 5, 7}

5: {5, 2, 4, 6, 8}

6: {6, 3, 5, 9}

7: {7, 4, 8}

8: {8, 5, 7, 9, 0}

9: {9, 6, 8}

Relación de recurrencia:
El caso base es:

$$T(1, d) = 1$$

porque para longitud 1 existe exactamente una combinación que termina en cada dígito: el propio dígito.

Para $k > 1$:

$$T(k, d) = \sum_{x \in Vecinos(d)} T(k-1, x)$$

Toda combinación válida de longitud k que termina en d se forma tomando una combinación válida de longitud k-1 que termina en un vecino x, y agregando d al final.

Supongamos que queremos contar correctamente las secuencias de longitud k que terminan en d.

Si alguno de los valores $T(k-1, x)$ estuviera mal calculado, entonces al sumarlo también quedaría mal calculado $T(k, d)$.

Por tanto, para obtener correctamente el conteo del problema grande, necesitamos que cada subproblema $T(k-1, x)$ esté correctamente resuelto.

El total de combinaciones de longitud n es:

$$Total(n) = \sum_{d=0}^{9} T(n, d)$$

Subproblemas traslapados:

Si implementamos la recurrencia de forma recursiva, el mismo estado $T(k, d)$ aparece muchas veces.

Por ejemplo, $T(4,5)$ puede ser necesario para calcular:

$$T(4,2), T(4,4), T(4,5), T(4,6), T(4,8)$$

porque desde 5 se puede mover hacia esos dígitos, o esos dígitos pueden depender de 5 como anterior.

Por tanto, los estados (k, d) se repiten en diferentes ramas de la recursión. Eso demuestra que hay subproblemas traslapados.

Una tabla DP evita esa repetición guardando cada valor $T(k, d)$ una sola vez.

Pseudocógido:

```
si n <= 0:
    retornar 0, []

combinaciones = ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]

para longitud = 2 hasta n:
    nuevas = []

    para cada numero en combinaciones:
        ultimo = último dígito de numero

        para cada siguiente en VECINOS[ultimo]:
            agregar numero + siguiente a nuevas

    combinaciones = nuevas
```

retornar cancombinaciones

Resultados:

```python
VECINOS = {
    0: [0, 8],
    1: [1, 2, 4],
    2: [2, 1, 3, 5],
    3: [3, 2, 6],
    4: [4, 1, 5, 7],
    5: [5, 2, 4, 6, 8],
    6: [6, 3, 5, 9],
    7: [7, 4, 8],
    8: [8, 5, 7, 9, 0],
    9: [9, 6, 8],
}


def contar_teclado(n):
    if n <= 0:
        return 0, []

    combinaciones = ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]

    for _ in range(2, n + 1):
        nuevas_combinaciones = []

        for numero in combinaciones:
            ultimo_digito = int(numero[-1])

            for siguiente in VECINOS[ultimo_digito]:
                nuevas_combinaciones.append(numero + str(siguiente))

        combinaciones = nuevas_combinaciones

    return len(combinaciones), combinaciones
```

```
Teclado Nokia

n = 2
Total de combinaciones: 36
Lista de combinaciones:
['00', '08', '11', '12', '14', '22', '21', '23', '25', '33', '32', '36', '44', '41', '45', '47', '55', '52', '54', '56', '58', '66', '63', '65', '69', '77', '74', '78', '88', '85', '87', '89', '80', '9
9', '96', '98']

n = 3
Total de combinaciones: 138
Lista de combinaciones:
['000', '008', '088', '085', '087', '089', '080', '111', '112', '114', '122', '121', '123', '125', '144', '141', '145', '147', '222', '221', '223', '225', '211', '212', '214', '233', '232', '236', '255
', '252', '254', '256', '258', '333', '332', '336', '322', '321', '323', '325', '366', '363', '365', '369', '444', '441', '445', '447', '411', '412', '414', '455', '452', '454', '456', '458', '477', '4
74', '478', '555', '552', '554', '556', '558', '522', '521', '523', '525', '544', '541', '545', '547', '566', '563', '565', '569', '588', '585', '587', '589', '580', '666', '663', '665', '669', '633',
'632', '636', '655', '652', '654', '656', '658', '699', '696', '698', '777', '774', '778', '744', '741', '745', '747', '788', '785', '787', '789', '780', '888', '885', '887', '889', '880', '855', '852'
, '854', '856', '858', '877', '874', '878', '899', '896', '898', '800', '808', '999', '996', '998', '966', '963', '965', '969', '988', '985', '987', '989', '980']

n = 4
Total de combinaciones: 532
Lista de combinaciones:
['0000', '0008', '0088', '0085', '0087', '0089', '0080', '0888', '0885', '0887', '0889', '0880', '0855', '0852', '0854', '0856', '0858', '0877', '0874', '0878', '0899', '0896', '0898', '0800', '0808',
'1111', '1112', '1114', '1122', '1121', '1123', '1125', '1144', '1141', '1145', '1147', '1222', '1221', '1223', '1225', '1211', '1212', '1214', '1233', '1232', '1236', '1255', '1252', '1254', '1256', '
1258', '1444', '1441', '1445', '1447', '1411', '1412', '1414', '1455', '1452', '1454', '1456', '1458', '1477', '1474', '1478', '2222', '2221', '2223', '2225', '2211', '2212', '2214', '2233', '2232', '2
236', '2255', '2252', '2254', '2256', '2258', '2111', '2112', '2114', '2122', '2121', '2123', '2125', '2144', '2141', '2145', '2147', '2333', '2332', '2336', '2322', '2321', '2323', '2325', '2366', '23
63', '2365', '2369', '2555', '2552', '2554', '2556', '2558', '2522', '2521', '2523', '2525', '2544', '2541', '2545', '2547', '2566', '2563', '2565', '2569', '2588', '2585', '2587', '2589', '2580', '333
3', '3332', '3336', '3322', '3321', '3323', '3325', '3366', '3363', '3365', '3369', '3222', '3221', '3223', '3225', '3211', '3212', '3214', '3233', '3232', '3236', '3255', '3252', '3254', '3256', '3258
', '3666', '3663', '3665', '3669', '3633', '3632', '3636', '3655', '3652', '3654', '3656', '3658', '3699', '3696', '3698', '4444', '4441', '4445', '4447', '4411', '4412', '4414', '4455', '4452', '4454'
, '4456', '4458', '4477', '4474', '4478', '4111', '4112', '4114', '4122', '4121', '4123', '4125', '4144', '4141', '4145', '4147', '4555', '4552', '4554', '4556', '4558', '4522', '4521', '4523', '4525',
'4544', '4541', '4545', '4547', '4566', '4563', '4565', '4569', '4588', '4585', '4587', '4589', '4580', '4777', '4774', '4778', '4744', '4741', '4745', '4747', '4788', '4785', '4787', '4789', '4780',
'5555', '5552', '5554', '5556', '5558', '5522', '5521', '5523', '5525', '5544', '5541', '5545', '5547', '5566', '5563', '5565', '5569', '5588', '5585', '5587', '5589', '5580', '5222', '5221', '5223', '
5225', '5211', '5212', '5214', '5233', '5232', '5236', '5255', '5252', '5254', '5256', '5258', '5444', '5441', '5445', '5447', '5411', '5412', '5414', '5455', '5452', '5454', '5456', '5458', '5477', '5
474', '5478', '5666', '5663', '5665', '5669', '5633', '5632', '5636', '5655', '5652', '5654', '5656', '5658', '5699', '5696', '5698', '5888', '5885', '5887', '5889', '5880', '5855', '5852', '5854', '58
56', '5858', '5877', '5874', '5878', '5899', '5896', '5898', '5800', '5808', '6666', '6663', '6665', '6669', '6633', '6632', '6636', '6655', '6652', '6654', '6656', '6658', '6699', '6696', '6698', '633
3', '6332', '6336', '6322', '6321', '6323', '6325', '6366', '6363', '6365', '6369', '6555', '6552', '6554', '6556', '6558', '6522', '6521', '6523', '6525', '6544', '6541', '6545', '6547', '6566', '6563
', '6565', '6569', '6588', '6585', '6587', '6589', '6580', '6999', '6996', '6998', '6966', '6963', '6965', '6969', '6988', '6985', '6987', '6989', '6980', '7777', '7774', '7778', '7744', '7741', '7745'
, '7747', '7788', '7785', '7787', '7789', '7780', '7444', '7441', '7445', '7447', '7411', '7412', '7414', '7455', '7452', '7454', '7456', '7458', '7477', '7474', '7478', '7888', '7885', '7887', '7889',
```